

**LABORATORIO DI PROGRAMMAZIONE
SICUREZZA DEI SISTEMI E DELLE RETI INFORMATICHE
UNIVERSITÀ DEGLI STUDI DI MILANO**
2023-2024
26.01.2024

INDICE

Parte 1 - [Linguaggio C]	1
Esercizio 1 - Le Otto Regine	1
Parte 2 - [Linguaggio C]	4
Esercizio 2 - Le Otto Regine - Funzioni	4
Esercizio 3 - Le Otto Regine - Main	5
Esercizio Bonus - Le Otto Regine	5
Soluzione	8
Errori Comuni	12

PARTE 1 - [LINGUAGGIO C]

Esercizio 1 - Le Otto Regine.

Ultima revisione: 7 febbraio 2024.

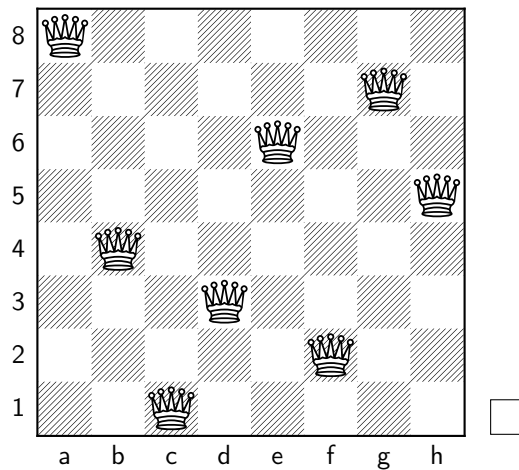


FIGURA 1. Una delle soluzioni al problema delle 8 regine.

Il **problema delle otto regine** è un rompicapo che consiste nel posizionare 8 regine, su di una scacchiera 8×8 , in maniera tale che nessuna possa catturarne un'altra. Una soluzione prevede quindi che nessuna regina abbia una colonna, una riga, o una diagonale in comune con una altra regina. Questo problema può essere generalizzato come problema delle n -regine; in questo caso, il problema sarà di disporre n regine su di una griglia $n \times n$ ¹.

Questo problema, semplice ma di non immediata soluzione, è stato spesso usato per sviluppare e testare algoritmi in diversi ambiti, da matematica applicata a ricerca operativa, ad intelligenza artificiale.

Questi algoritmi lavorano spesso in maniera iterativa o ricorsiva modificando lo stato corrente del sistema, al fine di avvicinarsi mano a mano alla soluzione. Uno stato del sistema è una configurazione della scacchiera, che potrebbe essere o non essere una soluzione. Per spostarsi tra uno stato e l'altro, l'algoritmo compie una *azione*, ovvero sposta una pedina da una posizione ad un'altra.

Un modo per rappresentare uno stato è un vettore, come nel caso qui sotto dove ogni elemento indica la riga in cui si trova la regina nella rispettiva colonna. Con questa rappresentazione, state facendo l'assunzione, valida, che ci sia obbligatoriamente una regina per ogni colonna. Ad esempio, per lo stato di Figura 1, una possibile rappresentazione è la seguente:

8	4	1	3	6	2	7	5
---	---	---	---	---	---	---	---

A partire da questa rappresentazione, è possibile costruire una matrice che rappresenti la scacchiera, se necessario. Su di una scacchiera, non vi è il vincolo che vi siano una e una sola regina per ogni colonna.

In questo esame dovrete implementare una serie di funzioni che permettano di verificare quanto uno stato della scacchiera sia vicino alla soluzione, affrontando il problema delle n regine con n fissato a tempo di compilazione.

Esercizio 1.1

Definite una funzione `stampa` che, ricevuto un vettore indicante uno stato della scacchiera, ed eventuali altri parametri necessari, stampi a schermo la rispettiva matrice indicando con il simbolo `-` una casella vuota bianca, con il simbolo `*` una casella vuota nera, e con il simbolo `Q` la presenza di una regina. Stampate anche le lettere indicanti righe e colonne, come in Figura 1.

Ad esempio, lo stato indicato in Figura 2 sarà stampato nel seguente modo:

8	-	*	-	*	-	*	-	*
7	*	-	*	-	*	-	*	-
6	-	*	-	*	-	*	-	*
5	*	-	*	Q	*	-	*	-
4	Q	*	-	*	Q	*	-	*
3	*	Q	*	-	*	Q	*	Q
2	-	*	Q	*	-	*	Q	*
1	*	-	*	-	*	-	*	-
	a	b	c	d	e	f	g	h

¹Consiglio: prima di affrontare il problema delle 8 regine direttamente, testate con dei casi più semplici il problema delle 4 regine.

8	18	12	14	13	13	12	14	14
7	14	16	13	15	12	14	12	16
6	14	12	18	13	15	12	14	14
5	15	14	14	♙	13	16	13	16
4	♙	14	17	15	♙	14	16	16
3	17	♙	16	18	15	♙	15	♙
2	18	14	♙	15	15	14	♙	16
1	14	14	13	17	12	14	12	18
	a	b	c	d	e	f	g	h

FIGURA 2. In questa figura è riportato il costo di ogni stato raggiungibile con una singola azione a partire dalla configurazione attuale. Ad esempio, il valore della prima casella A8, 18, è il costo della configurazione dove la prima regina è stata mossa da A4 ad A8.

Esercizio 1.2

Definite una funzione *verifica* che, ricevuto come input un vettore indicante uno stato della scacchiera, ed eventuali altri parametri necessari, restituisca 0 se lo stato è una soluzione al problema delle n regine, un numero > 0 se vi sono dei conflitti nella scacchiera. Un conflitto avviene se due o più regine si trovano sulla stessa riga, colonna, o diagonale.

Le ultime righe della parte di C del tema d'esame contengono dei suggerimenti e degli esempi che possono esservi utili.

Esercizio 1.3

Una funzione di costo *euristica* è una funzione che, dato uno stato, ci dice quanto sia vicino o lontano da una soluzione. Questo valore viene chiamato *costo*. Ad esempio, lo stato:

8	8	1	3	6	2	7	5
---	---	---	---	---	---	---	---

che è molto vicino alla soluzione di Fig. 1, avrà un costo basso. A partire da tale stato basterà fare pochi scambi per arrivare alla soluzione. Invece, uno stato come:

8	8	8	8	8	8	8	8
---	---	---	---	---	---	---	---

dove tutte le regine sono sulla stessa riga, sarà molto lontano dalla soluzione e quindi sarà avrà un costo alto.

4	3	2	5	4	3	2	3
---	---	---	---	---	---	---	---

Una funzione di costo euristica per il problema delle 8 regine si ottiene contando quante sono le coppie di regine che sono in conflitto. Ad esempio, per lo stato: ha un costo dato dalla funzione euristica di 17. Tale stato è mostrato in Figura 2. Un altro esempio: lo stato dove tutte le regine sono alla riga 8 ha un costo di 28. Una spiegazione di come calcolare l'euristica è data alla fine della Parte 2 del tema d'esame, con dei suggerimenti.

Definite una funzione `euristica` che, ricevuto come input un vettore indicante uno stato della scacchiera, ed eventuali altri parametri necessari, restituisca 0 se lo stato è una soluzione al problema delle 8 regine, il valore dell'euristica così definito in caso vi siano dei conflitti. Implementate eventuali altre funzioni necessarie.

Le ultime righe della parte di C del tema d'esame contengono dei suggerimenti e degli esempi che possono esservi utili.

Il main che utilizza la struttura dati così definita è oggetto della Parte 2. Testate le singole funzioni, e considerate i possibili errori che potrebbero verificarsi durante l'esecuzione.

PARTE 2 - [LINGUAGGIO C]

Esercizio 2 - Le Otto Regine - Funzioni.

Esercizio 2.1

Implementate una funzione `carica`, che riceve come input un nome di file e legge un array indicante uno stato, e lo usi per inizializzare lo stato corrente. In caso il file esista, assumete che sia strutturato in maniera corretta.

Esercizio 2.2

Implementate una funzione `cerca` che, ricevuto in ingresso lo stato corrente, una colonna in cui si vuole far spostare la regina, e la nuova posizione della regina (la nuova riga), restituisca il valore dell'euristica di tale stato, **senza modificare** lo stato corrente².

Ad esempio, dato lo stato corrente:

4	3	2	5	4	3	2	3
---	---	---	---	---	---	---	---

La chiamata `cerca(stato, 'g', 7)` valuterà lo spostamento della regina in colonna G in posizione G7, e deve restituire il valore 12, corrispondente allo stato:

4	3	2	5	4	3	7	3
---	---	---	---	---	---	---	---

Trovate come riferimento in Figura 2 l'euristica di tutti i possibili stati raggiungibili dalla stato usato come esempio qui.

²Se non avete fatto l'euristica, restituite il risultato della funzione `verifica`; se avete problemi a calcolare l'euristica o la funzione `verifica` contando le diagonali, fate una versione semplificata dove ipotizzate di avere delle torri invece che delle regine. Questa versione semplificata ovviamente verrà valutata in maniera diversa.

Esercizio 3 - Le Otto Regine - Main.

Scrivere un `main` che permetta all'utente di cercare manualmente la soluzione al problema delle n regine, con n definito a compile time. Il programma alla partenza chiederà un valore numerico all'utente e si comporterà nel seguente modo:

- 1 Stampa lo stato corrente della scacchiera;
- 2 Verifica e stampa a schermo se la scacchiera è una soluzione, o no;
- 3 Calcola con l'euristica il costo del corrente stato;
- 4 Carica da file una array a scelta dall'utente;
- 5 Chiede all'utente che mossa vuole fare, e restituisce il costo dello stato che si otterrebbe effettuando quella mossa, senza modificare lo stato corrente;
- 6 Chiede all'utente la mossa da fare, e modifica lo stato;
- 0 Esci dal programma

Per indicare una mossa, l'utente dirà la colonna su cui vuole fare l'operazione, e la riga dove verrà mossa la regina, assumendo che ci sia sempre una regina per ogni colonna. Ad esempio la mossa A8 metterà la regina della prima colonna nella prima casella in alto a sinistra. Il programma continuerà a chiedere una nuova istruzione valida all'utente in maniera continuata, **gestendo le eccezioni**.

Esercizio Bonus - Le Otto Regine.

Aggiungete al menù definito nell'Esercizio 3 la possibilità di stampare la matrice del costo di tutti gli stati raggiungibili con una sola azione, a partire dallo stato corrente. Tale matrice è rappresentata in Figura 2 e negli esempi riportati qui sotto. Attenzione a non modificare lo stato corrente.

Suggerimento 1

Per calcolare le diagonali, potete tenere conto che le diagonali inverse di una matrice sono uguali alle diagonali della matrice specchiata.

Ad esempio, in questo caso:

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

Le diagonali dirette sono 1-6-11-16; 5-10-15; 9-14; 13; 2-7-12; 3-8; 4.

La matrice specchiata sarà:

4	3	2	1
8	7	6	5
12	11	10	9
16	15	14	13

Le cui diagonali sono 4-7-10-13; 8-11-14; 12-15; 16; 3-6-9; 2-5; 1. Queste diagonali sono le diagonali inverse della matrice originale.

Suggerimento 2

In una colonna, riga, o diagonale dove ci sono $n > 1$ regine, il numero di conflitti è pari al numero di combinazioni semplici di 2 elementi, ovvero $\binom{n}{2} = n!/(2!(n-2)!)$. Si ricorda che il fattoriale è il prodotto dei primi n numeri, ovvero $4! = 4 * 3 * 2 * 1$, $3! = 3 * 2 * 1$.

Esempi

Per il problema delle 4 regine lo stato:

2	4	1	3
---	---	---	---

è una soluzione, così come lo stato:

3	1	4	2
---	---	---	---

Lo stato riportato qui sotto, non è una soluzione, e ha un costo di 6.

1	2	3	4
---	---	---	---

Trovate qui sotto la matrice che rappresenta il costo di ogni stato raggiungibile a partire dallo stato 1-2-3-4.

4	5	4	6
5	4	6	4
4	6	4	5
6	4	5	4

SOLUZIONE

Per risolvere il problema delle 8 regine, la cosa più semplice era calcolarsi una matrice di dimensione $N \times N$ a partire dallo stato, ed operare con una matrice. Vediamo per prima cosa la funzione stampa:

```
#include <stdio.h>
#include <stdlib.h>

#define N 4 // 8
#define MAXCHAR 'd' // 'h'
// Funzione per stampare lo stato della scacchiera
void stampa(int stato[N]) {
    for (int i = N; i > 0; i--) {
        printf("%d ", i);
        for (char j = 'a'; j < MAXCHAR+1; j=j+1) {
            if (i == stato[j-'a']) {
                printf(" Q ");
            } else if ((i + j) % 2 == 0) {
                printf(" * ");
            } else {
                printf(" - ");
            }
        }
        printf("\n");
    }
    printf(" ");
    for (int i=0; i<N; i++)
        printf(" %c ", 'a'+i);
    printf("\n");
}
```

Una volta che lo stato veniva convertito in una matrice, il problema diventava più semplice. Non potevano esserci conflitti sulle colonne, mentre era semplice verificare se vi erano conflitti sulle righe. Mentre per le diagonali, la rappresentazione matriciale aiutava a trovare una soluzione. Ve ne allego una di esempio.

```
int euristica(int stato[N]){
    int scacchiera[N][N];
    int reverseScacchiera[N][N];
    int conflitti = 0;
    int checkRiga, checkColonna, checkDiagonale, checkDiagonaleInversa;
    int i, j;

    // Creo la matrice
    for (i=0; i<N; i++)
        for (j=0; j<N; j++){
            scacchiera[i][j] = 0;
            // in reverseScacchiera metto la scacchiera specchiata
            reverseScacchiera[i][j] = 0;
        }

    for (i=0; i<N; i++){
```



```

        scacchiera[stato[i]-1][i] = 1;
        reverseScacchiera[stato[i]-1][N-i-1] = 1;
    }

    for (i=0; i<N;i++) {
        checkRiga = 0;
        checkColonna =0;
        for(j=0; j<N; j++){
            checkRiga += scacchiera[i][j];
            checkColonna += scacchiera[j][i];
        }
        if (checkRiga > 1)
            conflitti += disposizione(checkRiga,2);
        if (checkColonna > 1)
            conflitti += disposizione(checkColonna,2);
    }

    //controllo diagonale sx dx
    for (i=0; i<N;i++) {
        checkDiagonale = 0;
        checkDiagonaleInversa = 0;
        for(j=0; j<N-i; j++) {
            checkDiagonale+= scacchiera[i+j][j];
            checkDiagonaleInversa += reverseScacchiera[i+j][j];
        }
        if (checkDiagonale > 1)
            conflitti += disposizione(checkDiagonale,2);
        if (checkDiagonaleInversa > 1)
            conflitti += disposizione(checkDiagonaleInversa,2);
    }

    for (i=1; i<N; i++) {
        checkDiagonale = 0;
        checkDiagonaleInversa = 0;
        for(j=0; j<N-i; j++) {
            checkDiagonale+= scacchiera[j][j+i];
            checkDiagonaleInversa += reverseScacchiera[j][j+i];
        }
        if (checkDiagonale > 1)
            conflitti += disposizione(checkDiagonale,2);
        if (checkDiagonaleInversa > 1)
            conflitti += disposizione(checkDiagonaleInversa,2);
    }

    return conflitti;
}

// Non serviva fare due funzioni per verifica ed euristica, una sola bastava.
int verifica(int stato[N]){
    return euristica(stato);
}

```

}

La soluzione proposta non è immediata, e poteva creare qualche difficoltà. Era però possibile fare una soluzione concettualmente più semplice, anche se meno efficiente a livello di codice. Per ognuno degli $N \times N$ elementi della matrice, passo tutti gli $N \times N$ valori della matrice una seconda volta e controllo se sono sulla stessa diagonale.

```

int euristicaSemplice(int stato[N]) {
    int scacchiera[N][N];
    int reverseScacchiera[N][N];
    int conflitti = 0;
    int conflittiDiagonali = 0;
    int checkRiga, checkColonna, checkDiagonale, checkDiagonaleInversa;
    int i,j,k,l;

    // Creo una struttura dati che mi serva per
    for (i=0; i<N; i++){
        for (j=0; j<N; j++){
            scacchiera[i][j] = 0;
            reverseScacchiera[i][j] = 0;
        }

        for (i=0; i<N; i++){
            scacchiera[stato[i]-1][i] = 1;
            reverseScacchiera[stato[i]-1][N-i-1] = 1;
        }

        for (i=0; i<N; i++) {
            checkRiga = 0;
            checkColonna = 0;
            for (j=0; j<N; j++){
                checkRiga += scacchiera[i][j];
                checkColonna += scacchiera[j][i];
            }
            if (checkRiga > 1)
                conflitti += disposizione(checkRiga,2);
            if (checkColonna > 1)
                conflitti += disposizione(checkColonna,2);
        }

        // qui per ogni elemento della matrice effetto n^2 controlli.
        // questa soluzione controllera' n^4 valori, di una matrice
        // n x n
        for (i=0; i<N; i++)
            for (j=0; j<N; j++)
                for (k=0; k<N; k++)
                    for (l=0; l<N; l++) {
                        if ((i-k == j-l) && (i!=k)) {
                            if ((scacchiera[i][j]==scacchiera[k][l]) && (scacchiera[k][l] == 1))
                                {
                                    //printf("Conflitto nella diagonale da dx a sx\n");
                                    conflittiDiagonali+=1;
                                }
                        }
                    }
            }
    }
}

```

```

    }
}
if ((i-k == l-j) && (i!=k)) {
    if ((scacchiera[i][j]==scacchiera[k][l]) && (scacchiera[k][l] == 1))
    {
        //printf("Conflitto nella diagonale da sx a dx\n");
        conflittiDiagonali+=1;
    }
}
}
return conflitti+conflittiDiagonali/2;
}

```

Infine, era possibile calcolare l'euristica (e la funzione di verifica) direttamente dallo stato. Questa soluzione è concettualmente più difficile da pensare, ma molto più semplice ed elegante da implementare.

```

int euristicaConStato(int stato[N]) {
    int costo = 0;
    for (int i = 0; i < N; i++) {
        for (int j = i + 1; j < N; j++) {
            if (stato[i] == stato[j] || abs(i - j) == abs(stato[i] - stato[j])) {
                costo++;
            }
        }
    }
    return costo;
}

```

Riassumendo, c'erano tante vie per arrivare alla soluzione. Alcune più semplici da concettualizzare ma più verbose e meno efficienti da scrivere, altre più semplici ed eleganti ma anche meno immediate.

Per testare il lavoro man mano che lo sviluppate, è sempre meglio fare delle funzioni di test() con dei casi di base. Vediamo qui un esempio assieme ad una funzione che serviva per prendere i punti bonus.

```

void matriceEuristica(int stato[N]){
    int scacchiera[N][N];
    int i, j;
    int buffer[N];
    for (i=0; i<N; i++) {
        // copio lo stato
        for(j=0; j<N; j++)
            buffer[j] = stato[j];
        for(j=1; j<=N; j++){
            buffer[i] = j;
            printf("%d ", euristica(buffer));
        }
        printf("\n");
    }
}

```

```

int test() {
    // Vi conviene fare una funzione che vi permette di controllare
    // se i singoli step sono stati fatti in maniera corretta o meno
    // Inoltre, vi conviene usare come caso di base la matrice 4x4
    // che pi gestibile della 8x8.
    int statoOK1[N] = {2,4,1,3};
    int statoOK2[N] = {3,1,4,2};
    int statoKO1[N]  = {1,2,3,4};
    int statoKO2[N]  = {3,2,1,4};
    stampa(statoKO1);
    printf("verifica: %d %d ",verifica(statoKO1),euristica(statoKO1));
    printf("%d %d\n",euristicaConStato(statoKO1), euristicaSemplice(statoKO1));
    matriceEuristica(statoKO1);
    printf("\n");
    stampa(statoKO2);
    printf("verifica: %d %d ",verifica(statoKO2),euristica(statoKO2));
    printf("%d %d\n",euristicaConStato(statoKO2), euristicaSemplice(statoKO2));
    matriceEuristica(statoKO2);
    printf("\n");
    stampa(statoOK1);
    printf("verifica: %d %d ",verifica(statoOK1),euristica(statoOK1));
    printf("%d %d\n",euristicaConStato(statoOK1), euristicaSemplice(statoOK1));
    printf("\n");
    stampa(statoOK2);
    printf("verifica: %d %d ",verifica(statoOK2),euristica(statoOK2));
    printf("%d %d\n",euristicaConStato(statoOK2), euristicaSemplice(statoOK2));

    return 0;
}

int main(){
    test();
    return 0;
}

```

Per capire come lavorare con le matrici era comodo anche crearsi delle matrici con valori diversi da 0 e vedere se trovavate le diagonali in maniera corretta con delle `printf`

```

int testMatrix[N] = {{1,2,3,4},{5,6,7,8},{9,10,11,12},{13,14,15,16}};

```

ERRORI COMUNI

Per concludere, ecco una carrellata di errori tipici - ed evitabili - che sono stati commessi da più persone:

- Non usare la `define` per indicare il numero di regine e utilizzare una variabile o un valore hardcodato nel codice. Ad esempio, fare `int numregine = 8;` o peggio ancora `int stato[8];`
- Non testare gli algoritmi fatti su dei casi su dei casi semplici e noti, ma direttamente su delle matrici 8x8 inserite da utente;
- Non controllare che il file fosse aperto in maniera corretta.
- Non chiudere il file.

- Leggere dal file dei caratteri invece che degli interi. Leggere da file in maniera errata.
- Nella funzione cerca usare una malloc ma non fare la free corrispondente.
- Nella funzione cerca, modificare lo stato.
- Non usare lo switch nel menu ma fare tanti else-if annidati.
- Consegnare il file eseguibile ma non i sorgenti.
- Consegnare codice che non compila.